

Architecture overview

This document outlines the design and implementation of the Azure Cloud Assignment infrastructure. The environment was created using Infrastructure as Code (IaC) principles with Terraform, ensuring that deployments

- Version controlled
- Repeatable
- Consistent

Through Terraform, engineers can manage cloud resources declaratively, reducing manual configuration errors and ease collaboration across multiple teams.

The architecture is deployed into a single resource group (`rg-cloud-assignment`) located in the **North Europe** Azure region. Within this group, the environment provisions a Virtual Network, Subnet, Network Security Group (NSG), Public IP, Network Interface, a Linux Virtual Machine, and a Storage Account.

These resources form the foundation of a secure and functional cloud environment suitable for training and demonstration purposes.

Automation is achieved through a GitHub Actions pipeline, which integrates with Terraform to provide CI/CD capabilities.

This pipeline ensures that infrastructure changes are validated, planned, and applied consistently. Remote state management is configured via an Azure Storage Account, enabling collaborative workflows and preventing state drift.

Purpose and Scope

This environment is set to provide a reproducible and secure cloud infrastructure for educational and assignment use and can be expanded upon for a real internal tool.

What is included:

- A single Ubuntu Linux VM (`ubuntu-vm`) for application hosting and SSH access
- Networking components to isolate workloads and enforce security boundaries
- A storage account for both application data and Terraform state backend

- CI/CD automation via GitHub Actions to ensure repeatable deployments
- Documentation and a README to support handover to future engineers

Resource Inventory

The following resources are provisioned:

Resource Group

- `rg-cloud-assignment` in `northeurope`. Serves as the logical container for all resources. Due to VM availability in the Free Tier, I have attempted both `northeurope` and `westeurope`. Consistently there was an availability problem, and given this constraint, all other resources were successfully deployed but the VM
- An example can be found in this workflow:
<https://github.com/Momir9/azure-home-assignment/actions/runs/34836195443/job/103950465395>

Attempted instances:

Standard_A1_v2

Standard_A2_v2

Standard_A2m_v2

Standard_B1s

Standard_B2ats_v2

Standard_B2pts_v2

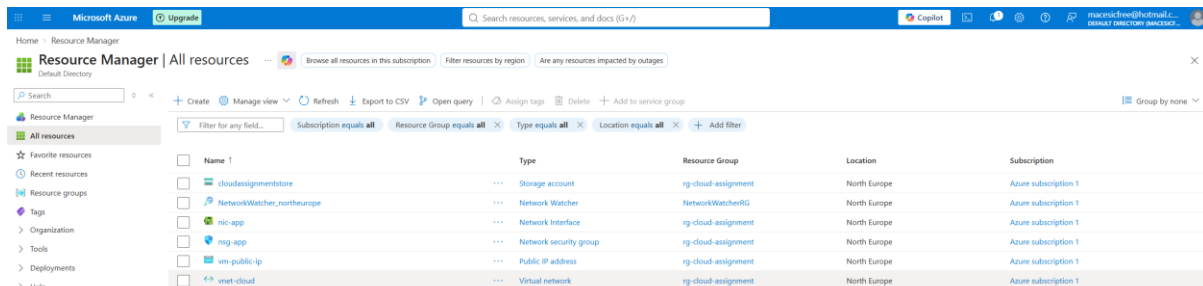
Standard_DS1_v2

Standard_D2s_v3

Standard_D2a_v4

Standard_D2as_v4

What i was able to deploy in earlier builds:



The screenshot shows the Azure portal interface. The left sidebar contains navigation links for Resource Manager, All resources, Recent resources, Favorite resources, Resource groups, Tags, Organization, Tools, Deployments, and Help. The main area displays a table of resources in the 'rg-cloud-assignment' resource group. The table has columns for Name, Type, Resource Group, Location, and Subscription. The resources listed are: cloudassignmentstore (Storage account), NetworkWatcher_northeurope (Network Watcher), nic-app (Network Interface), nsg-app (Network security group), vm-public-ip (Public IP address), and vnet-cloud (Virtual network). All resources are located in North Europe and belong to Azure subscription 1.

Name	Type	Resource Group	Location	Subscription
cloudassignmentstore	Storage account	rg-cloud-assignment	North Europe	Azure subscription 1
NetworkWatcher_northeurope	Network Watcher	NetworkWatcherRG	North Europe	Azure subscription 1
nic-app	Network Interface	rg-cloud-assignment	North Europe	Azure subscription 1
nsg-app	Network security group	rg-cloud-assignment	North Europe	Azure subscription 1
vm-public-ip	Public IP address	rg-cloud-assignment	North Europe	Azure subscription 1
vnet-cloud	Virtual network	rg-cloud-assignment	North Europe	Azure subscription 1

Networking components

- **Virtual Network (vnet-cloud):** Address space 10.0.0.0/16 which provides isolation
- **Subnet (app-subnet):** Address prefix 10.0.1.0/24 - hosts the VM and NIC
- **Network Security Group (nsg-app):** Inbound rule allowing TCP/22 only from ssh_source_ip
- **Public IP (vm-public-ip):** Static, Standard SKU, bound to NIC and enables external connectivity
- **Network Interface (nic-app):** Attached to the VM, connects to the subnet and NSG

Compute

- **Linux VM (ubuntu-vm):** Ubuntu 24.04 LTS image, size configurable via vm_size. Admin username azureuser. Provides compute for workloads.

Storage

- **Storage Account (cloudassignmentstore):** Standard LRS, StorageV2. Used for application data and Terraform backend. Ensures durability and availability.

Networking and Security Overview

Networking is designed to balance accessibility with security. The Virtual Network provides isolation, while the subnet hosts the VM and NIC.

NSG enforces strict inbound rules, allowing only SSH traffic from a whitelisted public IP. This prevents unauthorized access while enabling secure remote administration.

Outbound traffic is unrestricted, allowing the VM to get packages when needed.

The Public IP provides external connectivity, bound to the NIC, enabling SSH access.

Security controls extend beyond networking. The storage account enforces HTTPS traffic only, ensuring secure data transfer.

Terraform backend access is controlled via Azure RBAC, preventing unauthorized state modifications. Password authentication is enabled for simplicity, but the design supports SSH key authentication for stronger security.

Monitoring & Alerting

Monitoring and alerting are critical for operational visibility. In this environment, monitoring is implemented through Azure Monitor, with a specific focus on CPU utilization.

Metric Alert – CPU Usage

- A metric alert has been configured to trigger when the VM's average CPU utilization exceeds 80%.
- The alert is scoped to the VM resource (ubuntu-vm) and evaluates metrics every minute over a five-minute window.
- Severity is set to 2 (warning), ensuring that engineers are notified promptly without overwhelming them with false positives.

Action Group

- The alert is linked to an action group that sends notifications via email.
- This ensures that administrators are immediately informed when the VM experiences sustained high CPU usage.

While only CPU monitoring has been implemented, the design can be extended to include memory, disk, and network metrics. Future engineers can also integrate Log Analytics for deeper insights, enabling dashboards and advanced queries.

Deployment & Handover

Deployment follows a structured process:

1. Initialize Terraform backend (`terraform init -reconfigure`).
2. Plan infrastructure changes (`terraform plan -out=tfplan`).
3. Apply changes (`terraform apply -auto-approve tfplan`).
4. Validate outputs (`terraform output`).

Handover notes for future engineers:

- VM size may need adjustment based on regional availability.
- SSH access is restricted to `ssh_source_ip`. Update this variable as needed.
- Cleanup is achieved via `terraform destroy`.
- Monitoring currently includes only CPU alerts; future engineers should extend this to memory, disk, and network.
- Documentation and README are provided to ensure continuity.